INS Lab Exp-5

Aim:

Implementation of RSA Asymmetric key encryption algorithm.

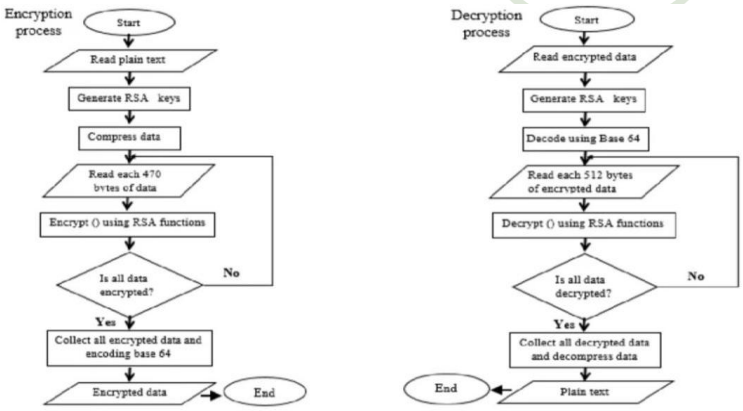
Description:

The RSA Cryptosystem is a widely used public-key (asymmetric) encryption algorithm that secures data transmission using a mathematically linked public and private key pair.
The process involves three main phases: key generation, encryption, and decryption.

1. Key Generation:
   - Choose primes: Select two distinct large prime numbers, p and q.
   - Calculate modulus (n): Multiply p and q to get $n = p \times q$. This value serves as the modulus for both keys.
   - Calculate totient ($\varphi(n)$): Compute Euler's totient function: $\varphi(n) = (p - 1) \times (q - 1)$.
   - Choose public exponent (e): Select an integer e such that $1 < e < \varphi(n)$ and e is coprime (shares no common factors other than 1) to $\varphi(n)$. The public key is the pair (n, e).
   - Calculate private exponent (d): Compute d as the modular multiplicative inverse of e modulo $\varphi(n)$, satisfying $(d \times e) = 1 \pmod{\varphi(n)}$. The private key is the pair (n, d).
2. Encryption:
   - The sender turns the plaintext message into a number m (where $m < n$).
   - The ciphertext c is calculated using the receiver's public key (n, e): $c = m^e \pmod{n}$.
3. Decryption:
   - The receiver takes the ciphertext c and applies their private key (n, d) to retrieve the original message m: $m = c^d \pmod{n}$.

The strength of RSA relies on the practical difficulty of factoring the product of two large prime numbers. While multiplying p and q to create n is computationally simple, reversing the process to find p and q from a massive number n is practically impossible with classical computers.

Flowchart:



 Signature of the Faculty.....

Implementation:

```
import math
p=7919
q=1009
n=p*q
phi=(p-1)*(q-1)
for e in range(2, phi):
    if math.gcd(e, phi)==1:
        break
for d in range(2, phi):
    if (e*d)%phi==1:
        break
print("Pubic key:", (e, n))
print("Private key:", (d, n))
m=int(input("Enter message(number): "))
c=pow(m,e,n)
print("Encrypted message:", c)
decrypted=pow(c,d,n)
print("Decrypted message:", decrypted)
```

Output:

```
Pubic key: (5, 7990271)
Private key: (1596269, 7990271)
Enter message(number): 432
Encrypted message: 7495573
Decrypted message: 432
```

Conclusion:

RSA encryption and decryption were successfully implemented using a pair of mathematically related keys. The experiment demonstrates the main advantage of asymmetric cryptography: the **public key can be used for encryption while the private key is required for decryption**. RSA obtains its security from the computational difficulty of factoring large numbers. RSA is useful for secure communication, authentication, digital signatures, and exchanging secret information. However, compared with symmetric algorithms, RSA requires more computation and is therefore not generally preferred for encrypting large amounts of data directly. The reference record also notes its computational limitations and potential future impact from advances in quantum computing.

Signature of the Faculty.....